

1. Testcase generator (Not JIRA)

The image displays two screenshots of a Visual Studio Code editor, illustrating the development of a Groq API client.

Top Screenshot:

- Explorer:** Shows the project structure for 'USER-STORY-TO-TESTS', including 'backend' (src, llm, groqClients, routes, generate.ts, prompt.ts, schemas.ts, server.ts) and 'frontend' (node_modules, src, components, apits, App.tsx, main.tsx, types.ts, vite-env.d.ts).
- Terminal:** Shows the command `npm run` being executed in the backend directory, resulting in a JSON response from the Groq API.
- Chat:** Displays a chat interface with a message: "Updating the JSON extractor to remove reliably and parse the first valid JSON block". The chat shows a "Fixed" status and a list of updates: "Updated groqClients.ts so extract now:". The updates include: "handles 'json ...' blocks even if the closing fence is missing", "strips stray leading/trailing backticks", and "returns only the first valid JSON object found".

Bottom Screenshot:

- Explorer:** Similar project structure, but the 'frontend' directory is expanded, showing 'package.json', 'tsconfig.json', 'tsconfig.node.json', 'vite.config.ts', and 'node_modules'.
- Terminal:** Shows the command `npm run` being executed in the frontend directory, resulting in a message: "VITE v5.4.21 ready in 1035 ms". Below this, the local and network URLs are displayed: "Local: http://localhost:5173/" and "Network: http://192.168.0.107:5173/".
- Chat:** Similar chat interface, but the message is: "Updating the JSON extractor to remove reliably and parse the first valid JSON block". The chat shows a "Fixed" status and a list of updates: "Updated groqClients.ts so extract now:". The updates include: "handles 'json ...' blocks even if the closing fence is missing", "strips stray leading/trailing backticks", and "returns only the first valid JSON object/array found".

User Story to Tests
Generate comprehensive test cases from your user stories

Story Title *

HC-310 — Doctor Prescription Verification Workflow

Description

The system should enable doctors to view medication details prescribed by other physicians or generated automatically from patient diagnosis. It should allow verification of dosage, potential interactions, and duplicate drugs. Once verified, prescriptions should automatically sync with the pharmacy module for patient dispensing.

Acceptance Criteria *

Then the system should highlight issues and provide AI-based recommendations for correction.
Given a verified prescription,
when it is approved,
Then it should automatically update the pharmacy dispensing queue.

Additional Info

Any additional information (optional)...

Generated Test Cases
20 test case(s) generated • Model: meta-llama/llama-4-scout-17b-16e-instruct • Tokens: 2815

Test Case ID	Title	Category	Priority	Expected Result
▶ TC-001	Doctor views pending prescriptions	Positive	High	Pending prescriptions are listed for verification
▶ TC-002	Prescription with errors is highlighted	Positive	High	System highlights issues and provides AI-based recommendations
▶ TC-003	Verified prescription updates pharmacy queue	Positive	High	Prescription is updated in pharmacy dispensing queue
▶ TC-004	Doctor views medication details	Positive	Medium	Medication details are displayed for review
▶ TC-005	Duplicate drug detection	Negative	High	System detects and highlights duplicate drug
▶ TC-006	Dosage conflict detection	Negative	High	System detects and highlights dosage conflict
▶ TC-007	AI-based recommendations for correction	Positive	High	System provides AI-based recommendations for correction

2. JIRA Integration:

Analyse my existing code base and Implement Jira integration
GO STEP BY STEP up on my approval proceed with next step
--[Mandatory] Backend first then proceed with Frontend

Instruction

[MANDATORY] Implement Jira connectivity email,api key should be configurable
[CRITICAL] Fetch the Projects from Jira (Projects should be autopopulated as dropdown)
[CRITICAL] Fetch User stories for the project (user stories should be autopopulated as dropdown)

tep-by-Step Implementation Plan

STEP 1: BACKEND - Jira Configuration & Connectivity

Create a new Jira client module to:

- Store configurable Jira URL, email, and API key in `.env`
- Authenticate with Jira REST API
- Test connectivity endpoint

What will be created:

- `backend/src/jira/jiraClient.ts` - Core Jira connectivity
 - `backend/src/routes/jira.ts` - Jira API endpoints
 - New `.env` variables: `JIRA_URL`, `JIRA_EMAIL`, `JIRA_API_KEY`
-

STEP 2: BACKEND - Fetch Jira Projects

Get projects from Jira and expose via API endpoint that frontend can call for dropdown population.

What will be created:

- API endpoint: `GET /api/jira/projects`
 - Returns: `{ projects: { key: string, name: string, id: string }[] }`
-

STEP 3: BACKEND - Fetch User Stories

Create endpoint to fetch user stories for a selected project.

What will be created:

- API endpoint: `GET /api/jira/issues?projectKey=KEY`
 - Returns: `{ issues: { key: string, summary: string, description: string }[] }`
-

STEP 4: BACKEND - Pre-populate Test Generation

Modify generate endpoint to accept Jira issue data instead of manual input.

What will be modified:

- `schemas.ts` - Add JiraIssue to GenerateRequest
 - `generate.ts` - Access Jira data
-

STEP 5: FRONTEND - Add Jira Configuration Form

Create settings/configuration panel for Jira credentials.

What will be created:

- `frontend/src/components/JiraConfig.tsx` - Configuration form
- Store credentials (securely in `env` or `localStorage`)

STEP 6: FRONTEND - Project Dropdown

Add project selector that calls backend `GET /api/jira/projects`.

What will be created:

- `frontend/src/components/ProjectSelector.tsx`

STEP 7: FRONTEND - User Stories Dropdown

Add user stories dropdown that populates based on selected project.

What will be created:

- `frontend/src/components/UserStorySelector.tsx`

STEP 8: FRONTEND - UI Integration

Integrate Jira selectors into main app, replace manual input with Jira data.

After many changes and vibe coding, finally output got generated...

From Jira Manual Input

Select Jira Project

myalproject (MYAIP)

Select User Story

MYAIP-25: Employee Profile Employee Self

Description:

As a user, I want to employee profile employee self so that the related functionality works as expected.

Story Title *

Employee Profile Employee Self

Description

From Jira Issue: MYAIP-25

Acceptance Criteria *

As a user, I want to employee profile employee self so that the related functionality works as expected.

